10TH CLASS COMPUTER SCIENCE — NEW SYLLABUS (PECTAA)

Chapter 4 — Control Structures in Python

Solved Exercise | New Book (PECTAA)

Exercise (Solved)

A. Multiple Choice Questions

1. What does the range() function do in Python?

(a) Generates a list of numbers
(b) Creates a sequence of numbers
(c) Calculates the sum of numbers
(d) Prints a range of numbers

Answer: (b) — The range() function creates a sequence of numbers which the for loop uses one by one. For example range(5) gives the numbers 0, 1, 2, 3 and 4. It does not sum the numbers and it does not print them by itself.

2. What is the output of the below code? count = 0; for i in range(2): for j in range(3): count += 1; print(count)

(a) 4
(b) 5
(c) 6
(d) 7

Answer: (c) — The outer loop runs 2 times and the inner loop runs 3 times for every turn of the outer loop. So count is increased 2 × 3 = 6 times, and the output is 6. This is the working of the nested loops.

3. Which method is used to add an item at the end of the list in Python?

(a) insert()
(b) append()
(c) add()
(d) extend()

Answer: (b) — The append() method adds a new item at the end of the list; for example dishes.append("Quorma"). The insert() method adds an item at a given position, and extend() joins another list at the end.

4. What is the purpose of control structures in programming?

(a) Only to make code longer
(b) To manage decision-making and repetition of tasks
(c) To create variables
(d) To import libraries

Answer: (b) — Control structures control the flow of a program. They are used to take decisions (if, if-else, if-elif-else) and to repeat the tasks (while and for loops), so a program can solve a problem in a proper order instead of running straight line by line.

studysapiens.com

5. Which statement is used to execute code when the condition is false?

(a) if (b) elif

(c) else (d) while

Answer: (c) — The if statement runs its block when the condition is True, while the else statement runs its block when the condition of the if statement is False. The elif is used for an additional condition.

6. How do you prevent print() from adding a newline?

(a) Use end="" (b) Use newline=False

(c) Use break (d) Use continue

Answer: (a) — The print() function adds a newline at the end of the output by default. If the parameter end="" is written, nothing is added at the end, so the next output continues on the same line.

7. Which library is used to generate random numbers?

(a) Datetime (b) statistics

(c) random (d) math

Answer: (c) — The random library is imported to generate random numbers, for example random.randint(1, 20). The statistics library is used for the mean, median and mode, the math library for mathematical functions, and the datetime library for dates and times.

8. What does list.append() do?

(a) Removes the first item (b) Adds an item to the end

(c) Sorts the list (d) Reverses the list

Answer: (b) — The append() method adds the given item at the end of the list and increases the length of the list by one. Removing an item is done with remove(), sorting with sort() and reversing with reverse().

9. The type of testing which checks how different parts of the code work together is:

(a) Unit Testing (b) Integration Testing

(c) Functional Testing (d) Regression Testing

Answer: (b) — Integration testing checks how the different parts of the code work together, that is, how the modules or functions are joined with one another. Unit testing checks the parts separately, functional testing checks the behaviour from the user's point of view, and regression testing checks that the new changes do not break the old working.

studysapiens.com

10. Which debugging technique involves adding output statements?

(a) Unit testing (b) Print statements

(c) Error messages (d) Integration testing

Answer: (b) — In this debugging technique the programmer adds extra print() statements inside the code to see the values of the variables and the flow of the program. When the error is found, these statements are removed.

B. Short Questions

1. Explain the use of the range() function in for loop?

The range() function creates a sequence of numbers, and the for loop uses this sequence to repeat its block of code. In every turn of the loop, the next number of the sequence is given to the loop variable. For example, range(5) gives 0, 1, 2, 3 and 4, so **for i in range(5)** runs five times; range(1, 11) gives the numbers from 1 to 10; and range(1, 11, 2) gives only 1, 3, 5, 7 and 9. In this way range() tells the loop how many times it has to run and which numbers it has to use.

2. What is the difference between the append and remove methods in Python lists?

append()	remove()
It adds a new item at the end of the list.	It deletes an item from the list.
The item is written inside the brackets, for example dishes.append(“Quorma”).	The item to be deleted is written inside the brackets, for example dishes.remove(“Nihari”).
The length of the list is increased by one.	The length of the list is decreased by one.
It is used when something new has to be added to the data.	It is used when an unwanted item has to be taken out of the data.

3. Define debugging.

Debugging is the process of finding, understanding and removing the errors (bugs) from a program, so that the program works correctly and gives the required output. In this process the programmer reads the error messages, tests the program with different inputs and checks the values of the variables. Debugging is done after the program is written, and it is a very important step because a program with errors cannot give the correct result.

4. What are the three parameters of the range() function?

The range() function can take three parameters:

(i) start: the number from which the sequence begins. If it is not given, the sequence begins from 0.

(ii) stop: the number at which the sequence stops; this number is not included in the sequence.

(iii) step: the amount by which the numbers increase or decrease. If it is not given, the step is 1.

For example, range(1, 10, 2) starts at 1 and stops before 10 with a step of 2, so it gives 1, 3, 5, 7 and 9.

5. Name three list methods and their purposes.

(i) append(): it adds a new item at the end of the list.

(ii) insert(): it adds a new item at a particular position (index) of the list.

(iii) remove(): it deletes the first matching item from the list.

Some other useful methods are **sort()**, which arranges the items in order, **pop()**, which removes the item of a given index, and **index()**, which tells the position of an item.

6. How do you import a library in Python?

A library is imported with the **import** statement, which is written at the top of the program. For example, **import random** imports the whole random library, and then its functions are used as **random.randint(1, 20)**. A single function can also be imported from a library, for example **from math import sqrt**, or the library can be given a short name, for example **import statistics as st**.

7. What is unit testing and why is it important?

Unit testing is a type of software testing in which the individual parts of the code, such as a function or a class, are tested separately and in isolation to check whether each part gives the correct result. In Python the unittest module is commonly used for this purpose. It is important because the errors are found at a very early stage, before the parts are joined together, so the repair becomes easy and cheap. When every unit works correctly, the whole program works better and there are fewer problems later.

8. How do you access the third element in a list?

The elements of a list are accessed with the help of an index written in square brackets, and the counting of the index starts from 0. So the first element is at index 0, the second at index 1 and the third at index 2. If the list is **dishes = [“Biryani”, “Nihari”, “Karahi”]**, then the third element is accessed as **dishes[2]**, which gives “Karahi”. In Python a negative index can also be used, so dishes[-1] gives the last element.

9. What is the purpose of the end parameter in print()?

By default the print() function puts a newline at the end of every output, so the next print() starts from a new line. The **end** parameter decides what is placed at the end of the printed text. If **end=""** is written, nothing is added at the end and the next output continues on the same line; if **end=" "** is written, one space is added. For example, print(i, end=" ") in a loop prints all the numbers in one line, separated by spaces.

10. How can we access an element from the list?

An element is accessed from a list by writing its index in square brackets after the name of the list. The index of the first element is 0, so **dishes[0]** gives the first element and **dishes[3]** gives the fourth one. A negative index counts from the end, so **dishes[-1]** gives the last element. A part of the list, called a slice, can also be taken, for example **dishes[1:3]** gives the elements from index 1 up to index 2.

11. How do you modify an existing item in a list?

An existing item of a list is modified by assigning a new value to the index of that item. For example, if the list is **dishes = [“Biryani”, “Nihari”, “Karahi”]**, then to change “Karahi” into “Chicken Karahi” we write **dishes[2] = “Chicken Karahi”**. The old value is replaced by the new one, and the length of the list remains the same.

C. Long Questions

1. Explain the different types of decision-making statements in Python with examples for each.

Decision-making statements are the control structures which let the program choose which block of code to run, according to a condition. Their main types in Python are the following.

(i) if statement: it runs its block only when the condition is True; if the condition is False, the block is skipped.

if statement

```
marks = 75
if marks &gt;= 40:
    print("Pass")
```

(ii) if-else statement: it gives two choices. The if block runs when the condition is True, and the else block runs when it is False.

if-else statement

```
marks = 35
if marks &gt;= 40:
    print("Pass")
else:
    print("Fail")
```

(iii) if-elif-else statement: it is used to check more than one condition in order. The first condition which becomes True is executed, and if none of them is True, the else block runs.

if-elif-else statement

```
marks = 85
if marks &gt;= 80:
    print("Grade A")
elif marks &gt;= 60:
    print("Grade B")
elif marks &gt;= 40:
    print("Grade C")
else:
    print("Fail")
```

(iv) Nested if statement: here one if statement is written inside another if statement. It is used when a condition has to be checked only after the first condition has become True.

Nested if statement

```
age = 20
citizen = "yes"
if age &gt;= 18:
    if citizen == "yes":
        print("You can apply for a licence")
    else:
        print("CNIC is required")
else:
    print("You are too young")
```

In all these statements the indentation (the space at the start of the line) is very important, because Python decides the block of code from the indentation.

2. Compare while loops and for loops.

while loop	for loop
It runs as long as the given condition remains True.	It repeats the block of code a specific number of times.
The condition is checked before every iteration, and the loop stops when the condition becomes False.	It takes the items of a sequence one by one, such as a list, a tuple, a string or a range of numbers.
It is used when the number of repetitions is not known in advance, for example waiting for the correct password.	It is used when the number of repetitions is known, for example printing the numbers from 1 to 10.
The loop variable is initialized before the loop and changed inside the loop; if it is forgotten, the loop may never stop.	The loop variable is initialized and changed automatically by the loop itself.
Example: **i = 1** **while i <= 5:** **print(i)** **i += 1**	Example: **for i in range(1, 6):** **print(i)**

Both loops can also be nested, that is, one loop can be written inside another loop. In that case, for every single turn of the outer loop, the inner loop completes all of its turns.

3. Describe how to work with Python lists, including creation and modification.

(i) Creating a list: a list is created by writing the items inside square brackets and separating them with commas. The items may be numbers, strings or a mixture of both, for example **dishes = [“Biryani”, “Nihari”, “Karahi”]** or **marks = [45, 67, 89]**.

(ii) Accessing the items: every item is accessed by its index, and the counting starts from 0, so dishes[0] is “Biryani” and dishes[2] is “Karahi”. A negative index

counts from the end, so dishes[-1] is the last item.

(iii) Adding an item: the **append()** method adds an item at the end of the list, and the **insert()** method adds an item at a particular position, for example **dishes.append("Quorma")** or **dishes.insert(1, "Haleem")**.

(iv) Modifying an item: a new value is assigned to the index of the item, for example **dishes[2] = "Chicken Karahi"** replaces "Karahi" with "Chicken Karahi".

(v) Removing an item: the **remove()** method deletes the given item, and **pop()** deletes the item of a given index, for example **dishes.remove("Nihari")**.

(vi) Other useful operations: len(dishes) tells the number of items, **dishes.sort()** arranges the items in order, and a for loop can be used to go through all the items one by one, for example **for d in dishes: print(d)**.

Program

```
dishes = ["Biryani", "Nihari", "Karahi", "Seekh Kababs"]
dishes.append("Quorma")                  # add at the end
dishes[2] = "Chicken Karahi"             # modify an item
dishes.remove("Nihari")                  # remove an item
print(dishes)
```

Output

```
['Biryani', 'Karahi', 'Seekh Kababs', 'Quorma']
```

4. Explain the importance of testing and debugging in programming. Discuss different debugging techniques.

Importance of testing: testing tells whether the program gives the correct output for the given inputs and whether it behaves as it is expected. It finds the errors before the program is given to the user, so the working of the software becomes reliable. The main types of testing are: **unit testing**, which tests the individual parts of the code in isolation; **integration testing**, which checks how the different parts work together; **functional testing**, which checks that the software behaves as expected from the user's point of view; and **regression testing**, which checks that the new changes do not affect the old working of the program.

Importance of debugging: debugging is the process of finding and removing the errors which were discovered during testing. Without debugging the program would keep on giving wrong results or would stop suddenly, so debugging makes the program correct and it also improves the understanding of the programmer about his own code.

Debugging techniques:

(i) Print statements: extra print() statements are added in the code to see the values of the variables at different steps, and when the error is found they are removed.

(ii) Using error messages: the message given by Python, such as NameError or TypeError, tells the line number and the type of the mistake, which helps in finding the error quickly.

(iii) Testing the parts separately: each function or small part of the code is run alone with different inputs, so the part which is giving the wrong result is found.

(iv) Step-by-step execution (dry run): the programmer goes through the program line by line and writes the value of each variable on paper, so he can see where the logic goes wrong.

(v) Using a debugger tool: the debugging feature of the IDE, such as PyCharm or VS Code, runs the program step by step and shows the values of the variables at every step.

5. How do Python libraries enhance programming efficiency? Provide an example.

A library is a ready-made collection of modules and functions which the programmer can use in his own program. Python offers an extensive standard library, and it is imported with the import statement. Libraries enhance the programming efficiency in the following ways.

(i) Time is saved: the programmer does not have to write the long code for common tasks himself; he simply calls the ready-made functions.

(ii) The code becomes short and easy: a difficult task, such as finding the mean of many numbers or generating a random number, is done in only one or two lines.

(iii) The code is tested and reliable: the functions of the standard libraries are already tested by the experts, so there are fewer errors in the program.

(iv) The programmer can focus on the real problem: instead of writing the tool, he works on solving the problem at hand.

Example: to generate random numbers and to find their mean, the **random** and **statistics** libraries are used, as shown below.

Program

```
import random
import statistics

numbers = []
for i in range(5):
    numbers.append(random.randint(1, 20))

print("Numbers:", numbers)
print("Mean:", statistics.mean(numbers))
```

Output (the numbers will be different every time)

```
Numbers: [7, 19, 3, 12, 15]
Mean: 11.2
```

Without these libraries the programmer would have to write his own random-number generator and his own formula for the mean, which would take much more time and could also contain mistakes.

6. Write a Python program using a loop that prints all the odd numbers between 1 and 20.

Program (method 1)

```
for i in range(1, 21, 2):
    print(i, end=" ")
```

Output

```
1 3 5 7 9 11 13 15 17 19
```

Here range(1, 21, 2) starts from 1, stops before 21 and moves with a step of 2, so only the odd numbers are produced. The same work can also be done by checking each number with the modulus operator:

Program (method 2)

```
for i in range(1, 21):
    if i % 2 != 0:
        print(i, end=" ")
```

If the remainder of a number after division by 2 is not zero (i % 2 != 0), the number is odd and it is printed.

7. Write a Python program that: (a) generates a list of 5 random numbers between 1 and 20, (b) uses the statistics library to calculate and print the mean of the generated numbers.

Program

```
import random
import statistics

numbers = []

for i in range(5):
    numbers.append(random.randint(1, 20))

print("Random numbers:", numbers)

mean = statistics.mean(numbers)
print("Mean of the numbers:", mean)
```

Output (the numbers will be different every time)

```
Random numbers: [14, 5, 18, 9, 11]
Mean of the numbers: 11.4
```

Explanation: the random library is imported to generate the random numbers with random.randint(1, 20), which gives a whole number between 1 and 20. An empty list is created first, and the loop appends five numbers into it. Then statistics.mean() is used to find the average of these numbers, which is printed on the screen.

8. Write a Python program that performs the following operations: (a) create a list of popular Pakistani dishes, (b) add a new dish “Quorma” to the list, (c) change “Karahi” to “Chicken Karahi”, (d) remove “Nihari” from the list.

Program

```
# (a) Create the list
dishes = ["Biryani", "Nihari", "Karahi", "Seekh Kababs"]
print("Original list:", dishes)

# (b) Add a new dish at the end
dishes.append("Quorma")
print("After adding Quorma:", dishes)

# (c) Change "Karahi" to "Chicken Karahi"
dishes[2] = "Chicken Karahi"
print("After changing Karahi:", dishes)

# (d) Remove "Nihari" from the list
dishes.remove("Nihari")
print("After removing Nihari:", dishes)
```

Output

```
Original list: [Biryani, Nihari, Karahi, Seekh Kababs]
After adding Quorma:
[Biryani, Nihari, Karahi, Seekh Kababs, Quorma]
After changing Karahi:
[Biryani, Nihari, Chicken Karahi, Seekh Kababs, Quorma]
After removing Nihari:
[Biryani, Chicken Karahi, Seekh Kababs, Quorma]
```

Explanation: append() adds the new dish at the end of the list, and the item at index 2, which is “Karahi”, is replaced by assigning a new value to it. Finally remove() deletes the item “Nihari” from the list.

9. Write a code to print stars (asterisk) in rectangular pattern using nested loops with three rows and five columns.

Program

```
for i in range(3):          # outer loop for the rows
    for j in range(5):      # inner loop for the columns
        print("*", end=" ")
    print()                 # move to the next line after each row
```

Output

```
* * * * *
* * * * *
* * * * *
```

Explanation: the outer loop runs 3 times, once for every row. For each turn of the outer loop, the inner loop runs 5 times and prints five stars on the same line, because end=" " stops the newline. After the inner loop finishes, the empty print() moves the output to the next line, so a rectangle of 3 rows and 5 columns is formed.